

DASE7506 MP1 Small Language Model Challenge Report

Modern GPT with Local Cache and Compact N Gram Backoff

Student	Course	Model freeze
[Li Qingchen] [3036746351]	DASE7506	26 September 2026

Abstract

This report presents a language model trained from random initialization on the supplied WikiText-2 training split. The final predictor combines an eight-block modern GPT, a strictly causal continuous cache confined to each 256-token evaluation window, an absolute-discount Kneser-Ney bigram distribution, and a compact discounted trigram backoff. Validation alone determines architecture checkpoints and mixture weights; the full test split is evaluated only after the method is frozen.

The frozen model achieves 1.5082599771 full-test bits per byte, compared with 2.1012602708 for the unchanged classroom baseline. This is a 28.22 percent relative reduction. The final evaluator takes 4.696 times the baseline CPU time, uses 1.877 GiB peak process memory, and carries 55.468 MiB of uncompressed inference assets, so it remains within all three assignment limits.

Table 1 Final result and constraint summary

Measure	Result	Limit or reference	Outcome
Full-test FP32 BPB	1.508260	Lower is better	Reported
Relative BPB reduction	28.22%	Baseline 2.101260	Improved
CPU time ratio	4.696x	At most 5x	Pass
Peak evaluation RAM	1.877 GiB	At most 4 GiB	Pass
Inference assets	55.468 MiB	At most 64 MiB	Pass

Keywords: causal language modelling, bits per byte, rotary position embeddings, continuous cache, Kneser-Ney smoothing, compact trigram model.

1 Assignment Requirements and Evaluation Protocol

The assignment ranks submissions by full-test bits per UTF-8 byte using the fixed BPE-2048 tokenizer, WikiText-2 files, evaluator, and independent causal windows of length 256. Models must use only the supplied training split, select settings on validation, avoid future-token access and cross-window state, and run within 5 times baseline CPU scoring time, 4 GiB peak RAM, and 64 MiB of inference assets.

The validation split contains 376,599 next-token targets and 1,148,007 raw bytes; the test split contains 428,405 targets and 1,292,013 bytes. All reported leaderboard and resource results use FP32 CPU evaluation with four PyTorch threads. The predictor was frozen before the first test evaluation, with checkpoint SHA-256 0f457cf814324e0b84ed9531eb1244ac0b764d3283f761dbdf48c3f9a27d7285.

2 Method

2.1 Modern GPT Backbone

The neural model has width 256, eight attention heads, eight transformer blocks, context length 256, and SwiGLU hidden width 704, for 6,951,168 trainable parameters. Rotary position embeddings replace learned absolute positions; RMSNorm replaces LayerNorm; SwiGLU replaces the baseline GELU feed-forward network; linear projections omit biases. Residual output matrices use depth-aware initialization, and dropout 0.15 regularizes the longer training run.

Table 2 Selected neural architecture and training settings

Component	Selected setting	Reason
Backbone	8 blocks width 256 8 heads	Increase capacity within the evaluation budget
Position and norm	RoPE and RMSNorm	Efficient relative position handling and stable residual computation
Feed-forward	SwiGLU hidden 704	Improve parameter efficiency over the baseline MLP
Optimization	AdamW BF16 seed 17	Stable from-scratch training on one GPU
Schedule	20,000 planned best at 19,000	Select the best checkpoint using validation only

2.2 Causal Local Cache and Count Backoff

At position t , the continuous cache compares the current hidden representation with earlier representations in the same evaluator window. A cached key at position i contributes the already observed successor token at $i+1$, and only $i < t$ is allowed. No cache state persists between model calls or windows. The selected cache uses similarity scale 10 and mixture weight 0.075.

$$p = 0.825 p_{\text{GPT}} + 0.075 p_{\text{cache}} + 0.025 p_{\text{bigram}} + 0.075 p_{\text{trigram}}$$

The count components use only 3,613,343 training tokens. The bigram distribution uses absolute-discount Kneser-Ney smoothing with discount 0.75. The trigram model uses discount 0.50 and stores at most eight continuations for each of 298,024 observed two-token contexts, with normalized backoff. This compact representation captures short lexical regularities while remaining below the asset limit.

3 Training and Experimental Design

3.1 Training Recipe

Training uses batch size 32, BF16 arithmetic, gradient clipping at 1.0, AdamW betas 0.9 and 0.95, matrix-only weight decay 0.1, 200 warmup steps, and cosine decay from learning rate 3.5e-4 to 5 percent of that value. Validation runs every 500 steps. The selected checkpoint is step 19,000, corresponding to 155,648,000 processed targets. Training takes 457.3 seconds; the complete process including preparation and validation takes 489.3 seconds on an NVIDIA GeForce RTX 5070 Ti.

3.2 Selection Discipline

1. Train candidate neural architectures from random initialization and retain each validation-best state.
2. Search neural-cache scale and weight on validation while reusing fixed neural weights.
3. Build bigram and trigram statistics from the supplied training split, then select mixture weights on validation.
4. Run causal and normalization tests, measure CPU time, memory, and assets, and freeze hashes before using test data.

3.3 Development Path

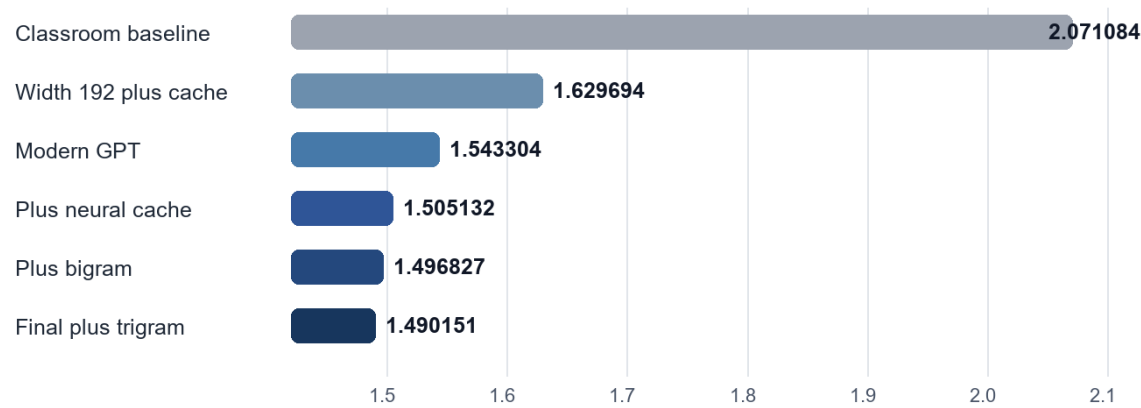


Figure 1 Validation BPB across the selected development path

The modern backbone and longer validation-selected training run produce the largest gain. The local cache then reduces validation BPB by 0.038172 with the neural weights and training targets held fixed. The count models provide smaller complementary improvements of 0.008305 and 0.006676 BPB.

Table 3 Model training runs

Run	Parameters	Selected targets	Best validation BPB	Process time
Classroom baseline	1.089M	9.830M	2.071084	369.2 s CPU
Modern 256 by 6	5.345M	126.976M	1.562021	308.5 s GPU
Modern 256 by 8	6.951M	155.648M	1.543304	489.3 s GPU

4 Controlled Comparisons and Ablations

4.1 Equal Training Target Comparisons

The assignment asks for a comparison at the same number of processed training targets. In each pair below, the cache and no-cache predictors share exactly the same trained neural parameters and target count. Only the causal prediction-time cache changes.

Table 4 Matched-weight cache comparisons

Fixed neural weights	Training targets	Without cache	With cache	BPB reduction
Classroom baseline	9,830,400	2.07108351	1.99864539	0.07243812
Modern final model	155,648,000	1.54330439	1.50513221	0.03817218

These comparisons isolate the benefit of the cache. They do not claim that the entire baseline-to-final improvement comes from caching, because the final backbone and training budget are also different.

4.2 Sequential Ablation of the Final Predictor

Table 5 Validation ablation with fixed final neural weights

Predictor	Validation BPB	Change from prior	Interpretation
Modern GPT only	1.543304	Reference	Neural backbone
Plus local neural cache	1.505132	-0.038172	Largest prediction-time gain
Plus Kneser-Ney bigram	1.496827	-0.008305	Adds local lexical frequency
Plus compact trigram	1.490151	-0.006676	Adds two-token context

4.3 Validation Search and Negative Results

For the bigram mixture, weight 0.05 gives 1.496827 BPB, while larger weights begin to degrade validation. Joint trigram search selects bigram weight 0.025 and trigram weight 0.075, producing 1.490152 BPB before the final CPU verification of 1.490151. These shallow optima support using small count-model weights rather than replacing the neural distribution.

Table 6 Rejected or limited alternatives

Experiment	Observed result	Decision
Information-weighted cache	Best search result remained above the selected linear cache	Reject
Low-rate continuation with EMA	Best validation 1.54348 versus parent 1.54330	Reject
Exact suffix cache	Improved 1.496827 to 1.493651 but raised runtime cost	Reject
Trigram discounts 0.75 and 0.90	Both were worse than discount 0.50	Reject

5 Full Test Result and Resource Compliance

The frozen predictor achieves 1.5082599771 test BPB. The unchanged classroom baseline achieves 2.1012602708 test BPB, giving an absolute reduction of 0.593000 and a relative reduction of 28.22 percent. Repeated test evaluation returns an identical BPB, so the reported score is deterministic under the stated CPU FP32 protocol.

Table 7 Final score and measured resource use

Metric	Measured result	Assignment limit	Status
Full-test BPB	1.5082599771	Lower is better	Reported
Baseline median CPU time	6.653 s	Reference	3 fresh runs
Final median CPU time	31.240 s	At most 33.263 s	4.696x pass
Peak process working set	1.877 GiB	At most 4 GiB	Pass
Uncompressed inference assets	55.468 MiB	At most 64 MiB	Pass

Timing uses three alternating fresh baseline and final evaluator processes on the same Ryzen 7 7800X3D with four PyTorch threads. Baseline times are 6.6526, 6.5938, and 7.0085 seconds; final times are 31.2403, 30.8348, and 34.0769 seconds. The ratio uses the two medians, which reduces sensitivity to one slow run. Peak RAM is the Windows process peak working set and includes imports, data, and checkpoint loading.

6 Critical Analysis

6.1 What the Evidence Supports

The experiments support three conclusions. First, the combined modern architecture and training recipe is substantially stronger than the original GPT configuration. Second, the causal local cache improves identical neural weights at equal processed-target counts in both the classroom and final models. Third, small bigram and trigram mixtures contribute additional information that the neural model and cache do not fully capture.

6.2 Limitations and Trade-offs

The modern-backbone comparison changes depth, normalization, position encoding, activation, dropout, optimizer settings, and training budget together, so it measures a recipe rather than an isolated architectural factor. Every trained model uses one random seed and one benchmark; small differences therefore lack statistical confidence intervals. Repeated validation searches may overfit the held-out split even though the test split was protected.

The final predictor consumes most of the time and asset budgets. Its 4.696x time ratio leaves little CPU margin, and its 55.468 MiB footprint uses 86.7 percent of the asset allowance. The evaluator resets context every 256 tokens, compact trigram storage discards rare continuations, and count models do not generalize semantically to unseen contexts. Further work should improve neural quality per unit of inference cost or replace dense count buffers with faster sparse structures.

7 Reproduction and Correctness

7.1 Evaluation Without Retraining

The repository contains the fixed evaluator, tokenizer, benchmark data, final self-contained student_v4 implementation, and the original contract test. The release asset contains checkpoint.pt. From the code directory, evaluate with:

```
conda create -n dase7506-mp1 python=3.12 pip -y
conda activate dase7506-mp1
python -m pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cpu
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python evaluate.py --checkpoint checkpoint.pt --device cpu --precision fp32 --threads 4 --split test
```

Expected full-test BPB is 1.5082599771. The checkpoint stores implementation student_v4 and all neural and count-model buffers, so evaluation requires no retraining or network access.

7.2 Correctness Checks

The original five contract tests check causality, probability normalization, batch independence, window reset behavior, gradient flow, and exact target coverage. The same tests also pass when directed to student_v4. Full validation evaluation of the self-contained final source reproduces 1.4901514899 BPB. These tests establish interface and causal correctness; they do not substitute for the held-out test score.

8 Attribution and AI Assistance

The course supplies the original GPT, trainer, tokenizer, data loader, evaluator, and WikiText-2 files. The continuous-cache design follows Grave, Joulin, and Usunier [1]. RoPE, RMSNorm, SwiGLU, and Kneser-Ney smoothing are implemented locally from standard descriptions. No pretrained weights, external training text, cached validation or test answers, or third-party model implementation is used.

OpenAI Codex provided assistance with testing, experiment execution, resource measurement, analysis, and report drafting.

References

- [1] E. Grave, A. Joulin, and N. Usunier. Improving Neural Language Models with a Continuous Cache. ICLR 2017. <https://arxiv.org/abs/1612.04426>
- [2] J. Su et al. RoFormer Enhanced Transformer with Rotary Position Embedding. Neurocomputing 2024. <https://arxiv.org/abs/2104.09864>
- [3] N. Shazeer. GLU Variants Improve Transformer. 2020. <https://arxiv.org/abs/2002.05202>
- [4] DASE7506 MP1 Small Language Model Challenge. Supplied GUIDE.md, evaluator, tokenizer, and WikiText-2 benchmark. Protocol 7506-mp1-wt2-v2.

Artifact Links

Code commit: <https://github.com/michaellleeoo/DASE7506-MP1/tree/4337cf678e13fdbdc55073f856195aba44303840>

Checkpoint: <https://github.com/michaellleeoo/DASE7506-MP1/releases/download/mp1-checkpoint/checkpoint.pt>